Building Your Own Security Lab

Training at Forrest.one · A Linux host with Kali and Windows, built to be broken



The Forrest.one bear mascot.

How to use this guide

This is a build manual. It takes a Linux machine you own and turns it into a lab you can attack, infect, snapshot and reset without risking anything real.

Six things to know before you start:

1. A lab is defined by what it cannot do. The isolation is the product, not the OS install.
2. Every layer has one trap that fails silently. This guide names them.
3. VMs and containers solve different problems. Use both.
4. Never work on something you cannot replace. Overlays and images are how you get that.
5. Verify isolation. Do not assume it. One command per side.
6. Build order matters. Each layer needs the one below it.

The rule that prevents every disaster in this guide

Never work on the thing you cannot replace.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

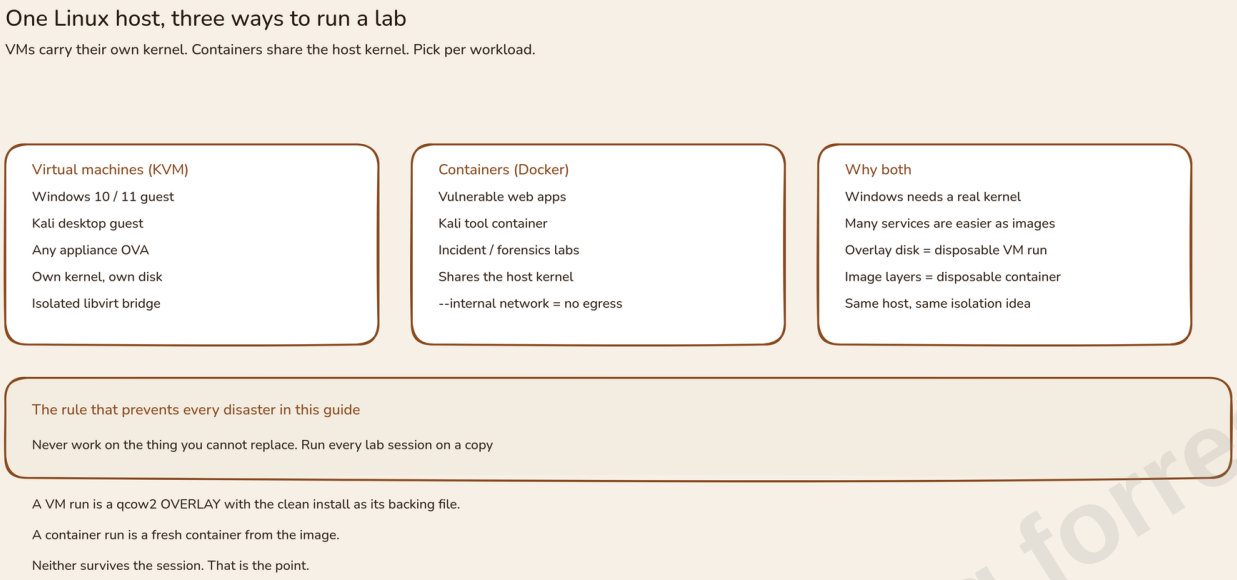


Figure 1: VMs, containers, and the disposable-run principle that applies to both.

What you build	Cost to keep it clean	The reset
A VM with a per-run overlay	One qemu-img create	Delete the overlay
A container from an image	One docker run	Remove the container
A bare-metal install	Hours, and it is gone	There isn't one

Everything below is built around the third row being something you never have to do.

1. What you are building

One Linux host running two kinds of machine:

Kind	Runs	Carries	Use it for
Virtual machine (KVM)	Its own kernel	Its own disk, network identity	Windows guests, Kali desktop, appliances
Container (Docker)	The host's kernel	Its own filesystem and network	Vulnerable web apps, tooling, ranges

A VM is a computer. A container is a process with boundaries. Windows needs a real kernel, so it is a VM. A deliberately-vulnerable web app is a process, so it is a container.

You want both, and they share one idea: start from something clean, work in a copy, throw the copy away.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

2. Prerequisites

Hardware

Requirement	Why	Check
64-bit CPU with VT-x (Intel) or AMD-V (AMD)	Hardware virtualisation	See below
Virtualisation enabled **in firmware**	A CPU that supports it may still have it off	BIOS/UEFI setting
16 GB RAM minimum, 32 GB comfortable	Each guest reserves its memory	free -h
100 GB free disk	Each VM is tens of GB	df -h
SSD strongly preferred	A VM on a spinning disk is painful	

Check the CPU and the kernel module before installing anything:

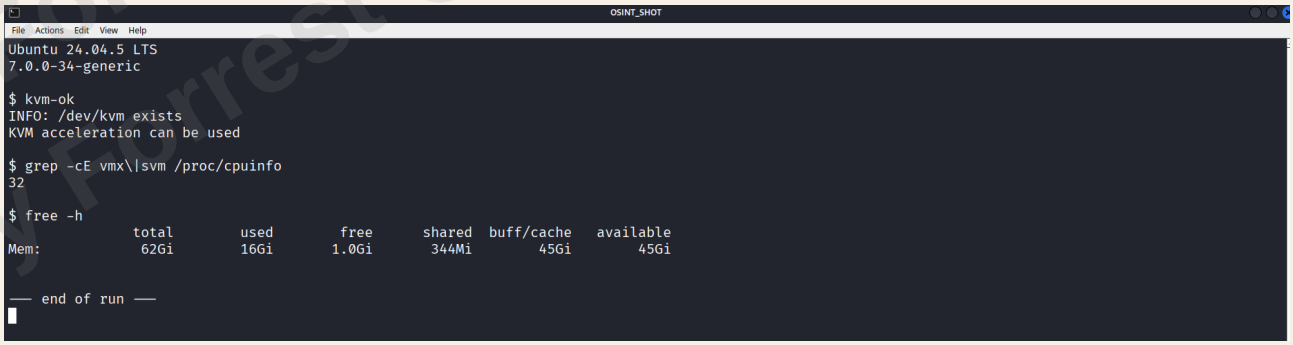
```
. /etc/os-release; echo "$PRETTY_NAME"
uname -r
grep -cE 'vmx|svm' /proc/cpuinfo      # non-zero = the CPU supports it
```



Screenshot: confirming the host OS, kernel, and CPU virtualisation support in the kernel.

32 matching processors is the CPU reporting virtualisation on each core.

```
kvm-ok
```



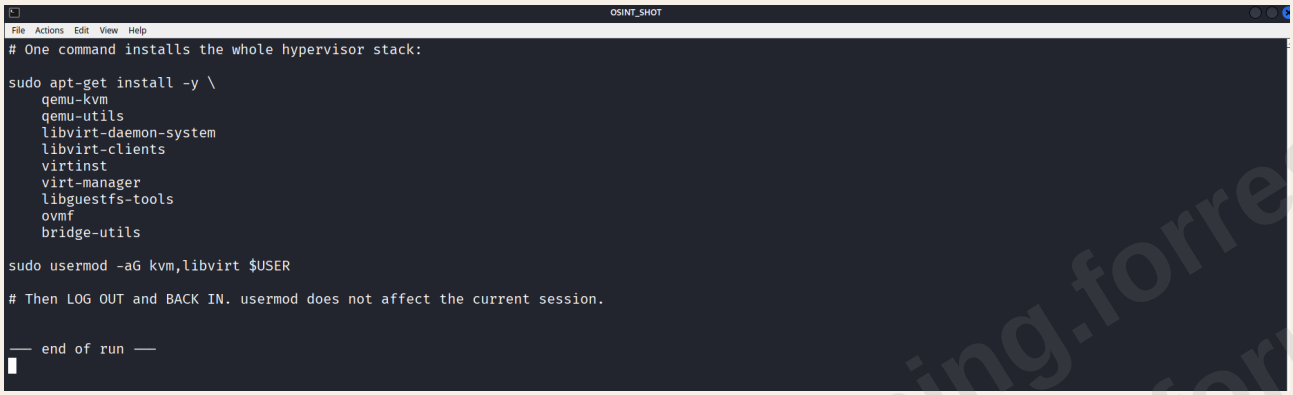
Screenshot: kvm-ok reporting that KVM acceleration can be used.

If kvm-ok says acceleration cannot be used, go into the firmware and enable virtualisation. No amount of software fixes that — the /dev/kvm device cannot work without it, and QEMU will silently fall back to emulation, which is dozens of times slower.

© Billy Forrest @ https://training.forrest.one

Software

```
sudo apt-get update
sudo apt-get install -y \
    qemu-kvm qemu-utils \
    libvirt-daemon-system libvirt-clients \
    virtinst virt-manager \
    libguestfs-tools ovmf bridge-utils
sudo usermod -aG kvm,libvirt "$USER"
```



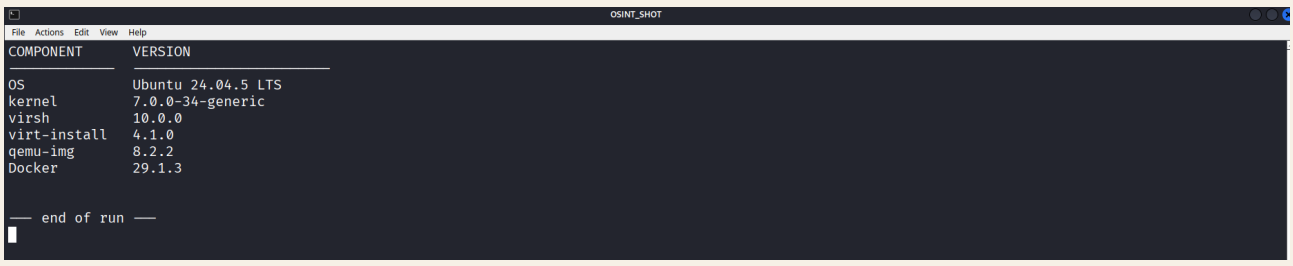
Screenshot: the single hypervisor install command and the group membership step.

That usermod line is not optional, and it is not instant. Group membership takes effect at the next login. Using the commands before logging out and back in produces permission errors that look like a broken install.

Package	Provides
qemu-kvm	The hypervisor that actually runs guests
qemu-utils	qemu-img — create, inspect and snapshot disks
libvirt-daemon-system	The libvirtd service and the system socket
libvirt-clients	virsh
virtinst	virt-install — build a domain in one command
virt-manager	The GUI, useful but not required
libguestfs-tools	Read a **stopped** guest's disk: virt-copy-out, virt-diff
ovmf	UEFI firmware for guests that need it
bridge-utils	Bridge inspection

Verify the whole stack

```
virsh --version
virt-install --version
qemu-img --version | head -1
docker --version
```



© Billy Forrest @ https://training.forrest.one

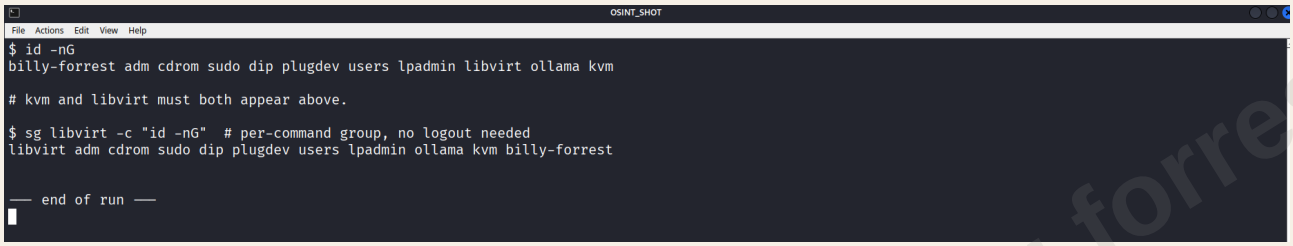
Screenshot: the versions of virsh, virt-install, qemu-img and Docker on the build host.

3. The permissions layer — where builds fail silently

This section saves more time than any other in this guide. Both traps produce errors that point somewhere else entirely.

Trap 1: the groups a normal shell does not carry

```
id -nG
```



Screenshot: checking which groups the current shell actually carries.

A new shell may not carry kvm and libvirt even after usermod. That is why every virsh command in this guide is wrapped in sg:

```
sg libvirt -c 'virsh -c qemu:///system list --all'
```

sg regenerates the group set for one command, so you do not have to log out.

Trap 2: sg takes ONE group, and nesting drops the outer one

This is the single most expensive trap in the setup. sg replaces the supplementary group set; it does not add to it.

Command	What you get
sg libvirt -c 'id -nG'	libvirt present, kvm absent
sg libvirt -c "sg kvm -c 'id -nG'"	**kvm present, libvirt GONE**
sg libvirt,kvm -c '...'	group 'libvirt,kvm' does not exist



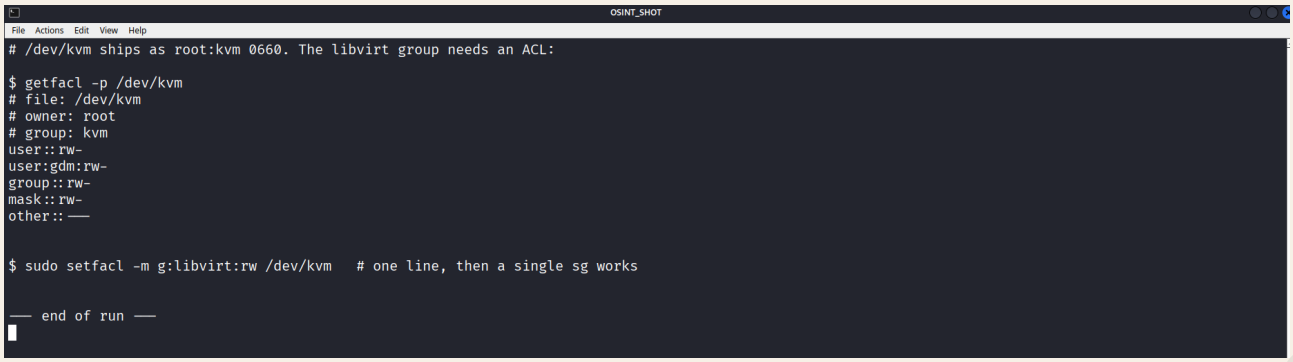
Screenshot: the nested-sg demonstration showing the outer group silently disappearing.

The nested form loses the outer group silently, so a command needing both the libvirt socket and /dev/kvm fails in a confusing way. The classic symptom: virt-install reports Network not found when the network exists — it reached the socket, but the daemon could not see the caller's group.

Fix it with an ACL on /dev/kvm, then use a single sg:

© Billy Forrest @ https://training.forrest.one

```
sudo setfacl -m g:libvirt:rw /dev/kvm
```



Screenshot: the /dev/kvm ACL, which lets one sg call open both the socket and the device.

/dev/kvm ships as root:kvm 0660. This one command gives the libvirt group access to it, is persistent, and removes the need for any group juggling.

Trap 3: qemu:///system runs the guest as a different user

qemu:///system launches QEMU as user libvirt-qemu (uid 64055), not as you. A disk under your home directory fails:

```
Cannot access storage file '/home/you/vm/disk.qcow2' (as uid:64055, gid:993): Permission denied
```

virt-install warns about this before it fails — that warning is the real blocker, not noise. libvirt-qemu needs only traverse (x) on each parent directory, not read:

```
sudo setfacl -m u:libvirt-qemu:x /home/$USER /home/$USER/labs
```



Screenshot: namei showing each path component's permissions down to the disk image.

Traverse-only is strictly better than loosening the directory mode — QEMU can walk to the disk but cannot list or read anything else in your home. Diagnose the whole chain at once with namei -l <path>.

Verify the ACL landed, because setfacl can no-op silently:

```
getfacl -p /home/$USER | grep libvirt-qemu
# expect: user:libvirt-qemu:--x
```

An empty grep means "not applied", not "applied differently".

© Billy Forrest @ https://training.forrest.one

The alternative: put VM assets where they belong

If ACLs are unavailable or you would rather not repeat them per project, use libvirt's own tree and skip the problem entirely:

```
sudo mkdir -p /var/lib/libvirt/images/<vm>
sudo mv base.qcow2 install.iso /var/lib/libvirt/images/<vm>/
sudo chown -R libvirt-qemu:kvm /var/lib/libvirt/images/<vm>
```

This is the standard layout and it is what most guides assume. The ACL route buys a self-contained working directory at the cost of two host-level grants.

4. Building a Linux or Kali VM

Get the ISO, and verify it

Kali publishes a checksum file next to the images. Use it — a torrent or mirror can hand you a truncated file that fails hours into an install.

```
curl -O https://cdimage.kali.org/current/kali-linux-2026.2-installer-amd64.iso
curl -O https://cdimage.kali.org/current/SHA256SUMS
sha256sum -c SHA256SUMS --ignore-missing 2>&1 | grep -i installer
```

Current release at the time of writing: Kali 2026.2, installer ISO 4,802,531,328 bytes, SHA-256 beginning 6dbefacc95e3b556.

Check any ISO cheaply before you build on it. file reports the ISO 9660 volume label and, crucially, whether the El Torito boot record is present:

```
file win10-ltsc-eval-x64.iso
# -> ISO 9660 CD-ROM filesystem data 'CESE_X64FREE_EN-US_DV9' (bootable)
```

The (bootable) suffix is the El Torito confirmation, and the quoted label is the authoritative edition check. A 4.9 GB file whose label is not the edition you asked for is a wrong download, not a corrupt one.

Build the domain

```
virt-install --name kali-vm --memory 4096 --vcpus 2 \
  --os-variant debian12 \
  --disk path=/var/lib/libvirt/images/kali-vm/base.qcow2,size=30,format=qcow2,bus=sata \
  --cdrom /var/lib/libvirt/images/kali-linux.iso \
  --network network=lab-isolated,model=e1000 \
  --machine q35 --cpu host-passthrough \
  --graphics vnc,listen=127.0.0.1,port=5901 --noautoconsole
```



Screenshot: the virt-install command that builds a VM from an ISO, headless.

© Billy Forrest @ https://training.forrest.one

Flag	Why it is set that way
--os-variant	Picks sane defaults for the guest type — always set it
bus=sata	Works on Windows without a driver (see §7)
--network network=lab-isolated	An isolating bridge, not NAT (see §5)
--graphics listen=127.0.0.1	**Never 0.0.0.0** — VNC has no authentication by default
--noautoconsole	Returns immediately; the guest keeps installing in the background

Get the --graphics choice right on the first attempt. Binding a console to 0.0.0.0 exposes keyboard control of a machine that will run hostile code, with no password.

Confirm it booted, and confirm which disk

```
sg libvirt -c 'virsh -c qemu:///system list --all'
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm'
```



Screenshot: virsh showing the domain running and which disk it has open.

--noautoconsole gives you no feedback, so prove the guest exists afterwards. Do not infer from CPU time — a VM can spin while sitting at a login screen.

See the guest without a VNC client

```
sg libvirt -c 'virsh -c qemu:///system screenshot kali-vm --file /tmp/vm.png'
```



Screenshot: capturing a guest's framebuffer from the host with virsh screenshot.

This is the fastest way to see what a guest is doing. It reads the hypervisor's framebuffer directly, so it needs no guest credentials, no network, and no viewer.

5. Networks — the isolation is the point

A lab machine that can reach the internet is a liability. It can exfiltrate, it can be attributed to you, and a sample that phones home has just told someone you are analysing it.

© Billy Forrest @ https://training.forrest.one

Isolation is a property you verify, not one you assume

A lab that can reach the internet is a liability, not a lab.

Docker: no egress

docker network create --internal lab-net

Internal=true, no gateway address

Containers have no route off the host

libvirt: no egress

Network XML with NO <forward> element

No NAT rule, no forwarding

Guest has no route off the host

How to PROVE it, on each side

Docker: docker network inspect lab-net -> Internal: true, Gateway empty

libvirt: virsh net-dumpxml lab-isolated | grep <forward> -> no output

Guest: try to resolve a name and watch it fail (that failure is the pass)

Never mark a lab ready on 'it came up'. Prove the isolation boundary.

The failure mode this prevents

A sample or a scanner that can reach the internet will reach it, and the only difference between a lab and an incident is who owns the target.

Isolate first, then build. Retrofitting isolation to a live lab means rebooting everything.

Figure 2: Isolation is a property you verify, not one you assume.

Three network shapes, and when to use each

The choice decides whether your lab can talk to the internet.

NAT bridge

default libvirt net

docker0 default bridge

Guest CAN reach internet

Use for: patching, installs

NOT for hostile code

Isolated bridge

No <forward> element

Own subnet, DHCP on host

No route off the host

Use for: detonation, ranges

Internal-only bridge

docker network --internal

No gateway at all

Containers see each other

Use for: attacker/victim pairs

Never put a detonation target on a NAT network

The whole observation you want is the C2 attempt FAILING. On a NAT network it succeeds, and you have run malware against the internet.

Give each range its own bridge so a reset of one cannot disturb another.

Name them for their purpose (lab-isolated, cpsa-net) so nobody has to guess later.

Figure 3: NAT, isolated and internal-only network shapes, and when to use each.

Docker: an internal network has no egress

```
docker network create --internal lab-net
docker network inspect lab-net --format '{{.Internal}} {{range .IPAM.Config}}gw={{.Gateway}}{{end}}'
```

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one



Screenshot: creating an internal Docker network and inspecting it.

--internal installs no gateway and no NAT rule, so a container on it has no route off the host. Containers on the same internal network still see each other, which is what makes attacker/victim pairs work.

libvirt: a network with no <forward> element

```
<network>
  <name>lab-isolated</name>
  <bridge name='virlab' stp='on' delay='0'/>
  <!-- Deliberately NO <forward> element: no NAT, no egress, no route off host. -->
  <ip address='192.168.200.1' netmask='255.255.255.0'>
    <dhcp><range start='192.168.200.100' end='192.168.200.200'/></dhcp>
  </ip>
</network>
```

```
virsh net-define lab-isolated.xml
virsh net-start lab-isolated
virsh net-autostart lab-isolated
```

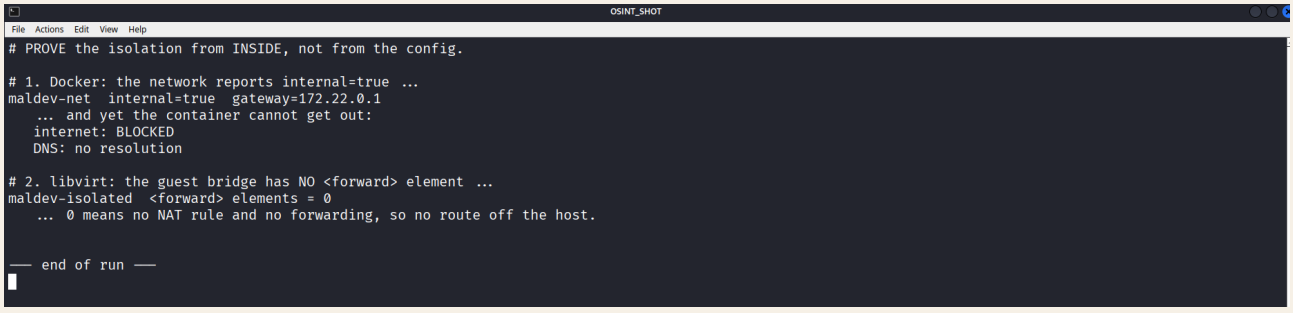
The absence of <forward> is the entire mechanism. Add one back and the guest gets NAT to the internet.

Prove the isolation — from inside, not from the config

Never mark a lab ready on "it came up". And do not prove isolation by reading the config alone — prove it from the guest or container itself.

```
# Docker: the network reports internal=true ...
docker network inspect lab-net --format '{{.Internal}} gateway={{range .IPAM.Config}}{{.Gateway}}{{end}
    }'
# ... and yet the container cannot get out:
docker exec victim sh -c 'timeout 5 nc -z 1.1.1.1 53 && echo REACHABLE || echo BLOCKED'
```

```
# libvirt: the guest bridge has no forward element at all
sg libvirt -c 'virsh -c qemu:///system net-dumpxml lab-isolated' | grep -c '<forward>'
```



© Billy Forrest @ https://training.forrest.one

Screenshot: proving isolation from inside the container and from the libvirt network XML.

Careful with the Docker case — this is a trap in itself. An internal network still reports a gateway address (in this lab, internal=true and gateway=172.22.0.1). The gateway exists on the bridge; what --internal removes is the iptables rule that forwards traffic out of it. So "there is no gateway" is the wrong test and would fail here.

Check	Isolated result
libvirt <forward> count	**0**
Container egress attempt	**BLOCKED**
Container DNS lookup	**no resolution**
Docker gateway address	**still present** — this proves nothing

The only test that counts is the outbound attempt failing. A working lab looks broken at this step. Expect "No Internet" inside a guest. Say so up front, or it reads as a build fault and you will waste an afternoon "fixing" the isolation you just correctly built.

6. Snapshots and overlays — making a run disposable

A container has disposability for free. A VM does not. Anything a sample drops — a driver, a scheduled task, a registry key, a startup entry — is still there at the next boot and contaminates the following session.



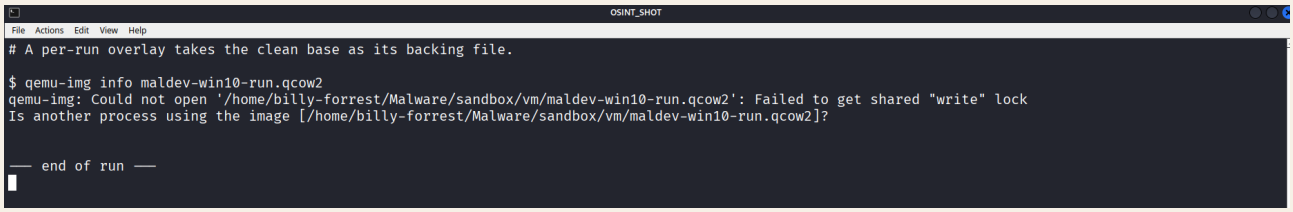
Figure 4: The base plus overlay pair, and how a run is discarded.

The pair

```
base.qcow2        the installed OS, written once, snapshotted "clean"
   └─ run.qcow2   qcow2 overlay with base as its backing file, created per run
```

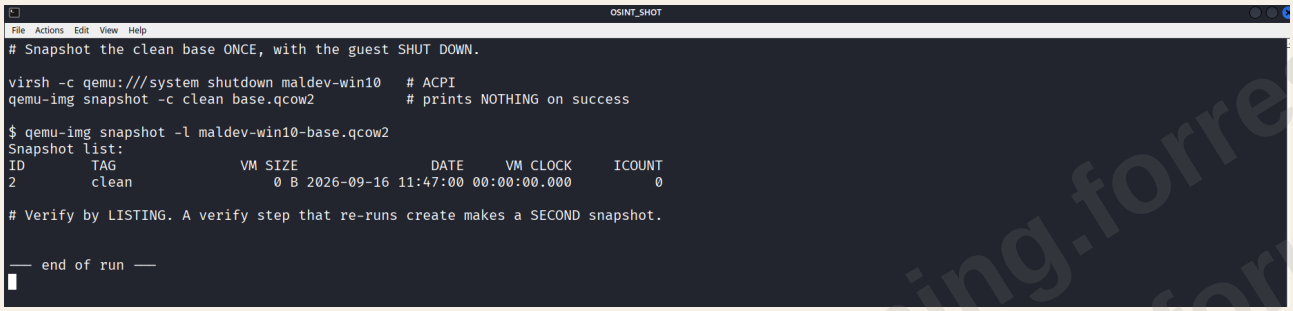
```
qemu-img create -f qcow2 -F qcow2 -b base.qcow2 run.qcow2
```


© Billy Forrest @ https://training.forrest.one



Screenshot: qemu-img info confirming the overlay's backing file.

```
qemu-img snapshot -c clean base.qcow2    # record the revert point
qemu-img snapshot -l base.qcow2          # LIST it (see the trap below)
```



Screenshot: listing the base image's snapshot to verify it exists.

Snapshot with the guest SHUT DOWN

Two separate blockers stop a snapshot on a running VM, and they look like one problem:

```
qemu-img: Could not open 'base.qcow2': Permission denied
qemu-img: Could not open 'base.qcow2': Failed to get shared "write" lock
```

QEMU holds the image while the guest runs, and it owns the file while doing so. A clean ACPI shutdown releases both:

```
virsh -c qemu:///system shutdown kali-vm     # ACPI: the guest closes properly
# wait for virsh list --all to show "shut off" (~30s for Windows)
qemu-img snapshot -c clean base.qcow2
```

Three traps in the snapshot step

Trap	Why it bites
qemu-img snapshot -c **prints nothing on success**	... || echo failed never fires, so a failed snapshot looks identical to a good one
Verifying by **re-running** the create	Silently makes a **second** snapshot under the same name
Snapshotting a **running** guest	Fails on the lock, and the error reads like a permissions problem

Verify by listing, never by re-creating.

The most dangerous state in the whole setup

A domain that reports clean while writing directly to the base disk.

```
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm'
```

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one



Screenshot: domblklist showing the domain is on the overlay, not the base.

domblklist shows	Meaning
sda -> .../run.qcow2	**Correct** — a disposable run
sda -> .../base.qcow2	**Contaminating** — stop and repair before detonating

This is why "there is no overlay file" is not evidence of a clean state. With no overlay, every write goes permanently into the base. Ask the running domain which disk it has open, because that is the only check that reflects reality.

Attaching the overlay reliably

virsh update-device --live cannot swap a root disk, and it fails silently in the most dangerous direction: the overlay file gets created, the command reports only a soft error, and the guest boots on the base.

Patch the <disk> element in the definition and redefine it instead. Two details decide whether it works:

1. Omit <backingStore/> entirely. Leaving it empty alongside an overlay <source> makes libvirt believe the overlay has no parent, and the start fails with a backing-file mismatch that reads like a corrupt image.
2. Keep <target> and <address> byte-identical. Change them and the guest's boot order shifts.

Then verify twice — config and live state:

```
virsh dumpxml kali-vm | grep '<source file'   # the definition points at the overlay
virsh domblklist kali-vm                      # the LIVE device is the overlay
```

Refuse to proceed if either disagrees.

Reset: drop the overlay and verify it went

```
rm -f run.qcow2
[ -f run.qcow2 ] && echo "restore FAILED: overlay still present"
```

rm -f returns success even when nothing was deleted. A reset that trusts it can report "restored" while the sample's changes are still live. Point the definition back at the base before deleting the overlay — deleting first leaves the domain referencing a missing disk, and the next start fails with an unrelated-looking error.

7. Windows guests — four decisions

Windows does not ship virtio drivers, and that single fact drives three of these.

© Billy Forrest @ https://training.forrest.one

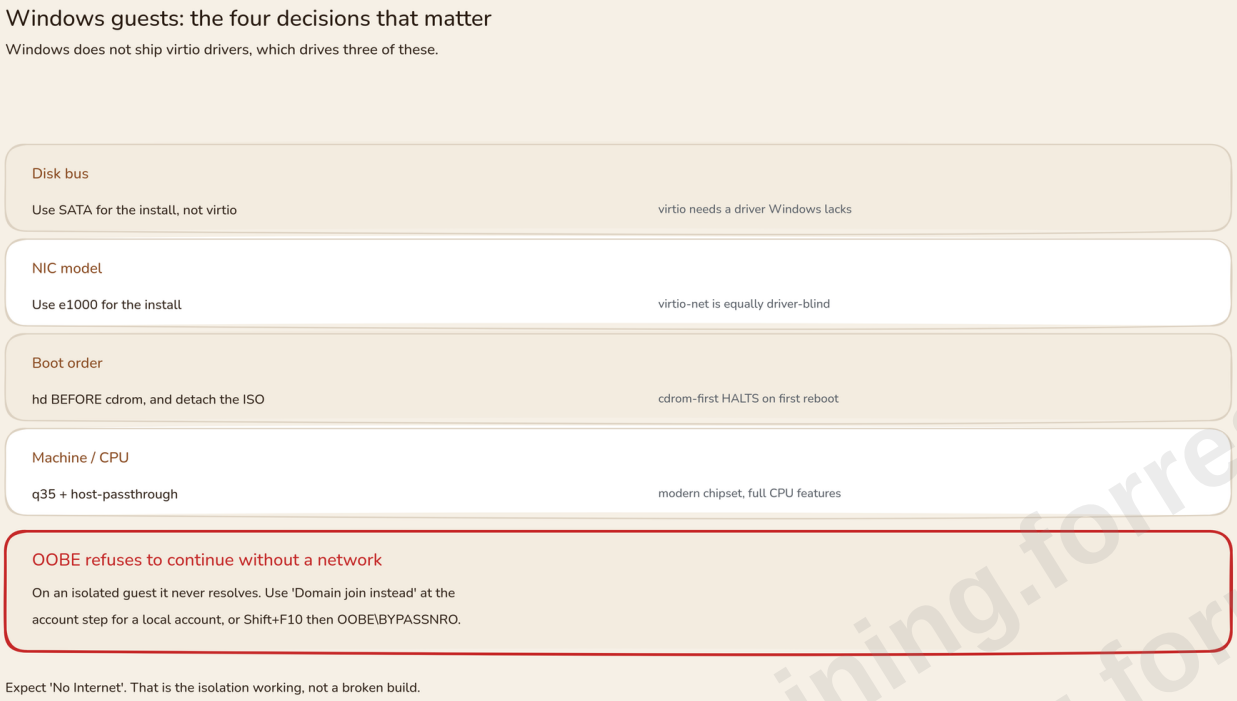


Figure 5: The four Windows guest decisions, and the reason for each.

Decision	Use	Because
Disk bus	bus=sata for the install	virtio needs a driver Windows does not have
NIC model	model=e1000 for the install	virtio-net is equally driver-blind
Boot order	hd **before** cdrom, and detach the ISO	cdrom-first **halts** on the first reboot
Machine / CPU	--machine q35 --cpu host-passthrough	Modern chipset, full CPU feature set

The boot-order trap, and how it disguises itself

virt-install --boot cdrom,hd writes <boot dev='cdrom'/> before <boot dev='hd'/>. That is correct for the install and wrong the moment it ends: on the first post-install reboot the guest hits "Press any key to boot from CD", no key arrives, and it halts.

The symptom triad reads as a console problem, and it is not:

You observe	Reality
virsh list --all shows shut off	The guest halted itself
A disk image of several GB	The install **did** succeed
The console page loads, then cannot connect	Nothing is listening on the VNC port

Fix both halves — reorder the boot devices and remove the ISO, so there is nothing but the OS to boot from:

```
sg libvirt -c 'virsh -c qemu:///system dumpxml kali-vm' > dom.xml
# edit: <boot dev='hd'/> before <boot dev='cdrom'/>; delete the <disk ...cdrom> element
sg libvirt -c 'virsh -c qemu:///system define dom.xml'
sg libvirt -c 'virsh -c qemu:///system domblklist kali-vm'   # only the OS disk remains
```

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

OOBE will refuse to continue without a network

Windows setup will not proceed when it cannot reach the internet — and on an isolated guest it never will. Two ways past it, in order of preference:

1. "Domain join instead" at the account step — creates a local account, no Microsoft login, no telemetry account.
2. Shift+F10 for a command prompt, then run OOBE\BYPASSNRO, which reboots and adds an "I don't have internet" option.

Expect the guest to show "No Internet" / an unidentified network throughout. That is the isolation working. It is not a broken build.

Confirm the guest really booted

```
sg libvirt -c 'virsh -c qemu:///system screenshot win10 --file /tmp/win.png'
```



Screenshot: the Windows guest's framebuffer captured from the host, showing it booted.

This capture was taken from the host with no guest credentials and no VNC client — which is the whole argument for driving the hypervisor rather than the guest.

8. The container half of the lab

Not every lab needs a VM. A vulnerable web app, a tooling container, or an attacker/victim pair is much faster as a container.

A single internal network

```
docker network create --internal lab-net
docker run -d --name victim --network lab-net alpine:3.20 sleep infinity
docker run -d --name attacker --network lab-net alpine:3.20 sleep infinity
```

Docker	Equivalent libvirt
docker network create --internal	a network with no <forward> element
--cap-drop ALL --security-opt no-new-privileges	a hardened guest
docker rm -f <container>	deleting the overlay
an image	base.qcow2

Hardening a container, and what it costs you

```
docker run -d --name analyst \
  --network lab-net --hostname WS-01 \
  --cap-drop ALL --cap-add NET_RAW \
  --security-opt no-new-privileges \
  --pids-limit 512 --memory 4g --cpus 2 \
  --tmpfs /tmp:rw,nosuid,nodev,size=512m \
  image sleep infinity
```


© Billy Forrest @ https://training.forrest.one

--cap-drop ALL plus --security-opt no-new-privileges is strong, and it breaks tcpdump.

Flag	Effect
--cap-drop ALL	Removes every capability from the container
--security-opt no-new-privileges	Forbids raising privileges back — **including via file capabilities**
Result	tcpdump exits Operation not permitted, because it needs a capability raise to apply its own file caps

Do not "fix" this by dropping no-new-privileges. That is a genuine security loss. Capture at the syscall layer (strace) instead, which for malware work is the better signal anyway — it records the process name alongside the connection attempt.

A cap that cannot be re-added

--cap-add NET_RAW looks like it solves the capture problem and does not. With no-new-privileges in effect, CapEff stays zero and tcpdump still fails. The flag and the capability are in direct conflict, and the security flag should win.

9. Putting it together



Figure 6: The eight layers, in the order you install them.

Layer	Do this	The trap
1 Hardware	Enable VT-x / AMD-V in firmware	kvm-ok fails and you blame the OS
2 Host OS	Ubuntu 24.04 or Kali, bare metal	—
3 Virtualisation	The one apt-get install	usermod needs a fresh login

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

Layer	Do this	The trap
4 Permissions	kvm + libvirt groups, /dev/kvm ACL, home traverse ACL	Nested sg drops a group **silently**
5 Networks	Isolated bridge, no <forward>; --internal for Docker	A lab that reaches the internet
6 Storage	Base + overlay, snapshot the base while shut down	Writing to the base and not noticing
7 Guests	Windows with SATA + e1000, hd before cdrom	The install halts on first reboot
8 Container ranges	Compose on an internal network	tcpdump broken by the hardening

The first-build cost is real

Task	Roughly
apt-get install the hypervisor stack	2–5 minutes
Windows install into a qcow2	20–40 minutes
Kali install from ISO	15–30 minutes
A container image pull	1–10 minutes
Every build after that	**seconds** — overlays and cached images

Nothing in this guide is slow the second time. That is the return on building it properly.

10. Troubleshooting

Every one of these was hit while building this lab.

Symptom	Cause	Fix
Network not found from virt-install, but the network exists	Nested sg dropped the libvirt group	setfacl -m g:libvirt:rw /dev/kvm, then a **single** sg
Cannot access storage file ... (as uid:64055)	libvirt-qemu cannot traverse your home	setfacl -m u:libvirt-qemu:x on each parent dir
Permission denied opening a qcow2 for snapshot	The guest is still running	ACPI shutdown, then snapshot
Failed to get shared "write" lock	Same — QEMU holds the image	Stop the guest
Guest halts on first reboot after install	cdrom before hd in boot order	Reorder and detach the ISO
Console page loads but will not connect	Nothing is listening on the VNC port	The guest is not running — check virsh list
qemu-img snapshot -c printed nothing, did it work?	It prints nothing on success **by design**	Verify with qemu-img snapshot -l
Two snapshots with the same tag	A "verify" step that re-ran the create	List; delete the duplicate with snapshot -d
tcpdump: Operation not permitted in a container	no-new-privileges blocks the capability raise	Capture with strace; keep the flag
Windows OOBE will not continue	No internet on an isolated guest	"**Domain join instead**", or Shift+F10 then OOBE\BYPASSNRO
Guest "clean" but changes persist	The domain is on base.qcow2	domblklist; reattach the overlay
supermin exited with error status 1	/boot/vmlinuz-* is 0600 root:root, unreadable	ls -l /boot/vmlinuz-* then sudo chmod 644

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one

The supermin one deserves a note. It breaks virt-copy-out and virt-diff, and it is almost never the appliance's fault. Check /boot permissions before anything else — a reboot does not change file modes, so if that is the cause, rebooting costs you a full stop of every VM and container for nothing.

And when every extraction fails, that is tooling, not an empty guest. Report the distinction: one absent path means the sample wrote nothing there; all four failing means the appliance never built, and reporting "not present" for each would be a false negative.

11. Working with a stopped guest

You can read a guest's disk without booting it or knowing any credentials.

```
virt-copy-out -a run.qcow2 /Windows/Temp ./evidence/
virt-diff -a base.qcow2 -A run.qcow2 > FILESYSTEM-DIFF.txt
```

Tool	Gives you	Needs
virt-copy-out	Files from a stopped disk	libguestfs-tools
virt-diff	**Every file created, modified or deleted** since the base	nothing else
virt-ls, virt-cat	Listings, single files	nothing else

virt-diff is the dynamic-analysis workhorse. It needs no guest tooling, no credentials and no symbols, works on the stopped overlay, and covers paths a fixed collection list would miss.

```
= - 0777    8192 /path/changed      size or content changed
+ - 0777       0 /path/created      present only in the overlay
- - 0777       0 /path/deleted      present only in the base
```

Expect Windows' own telemetry to dominate a boot-only diff — Defender, DiagTrack, Search. That noise is the baseline, not a finding. Say which is which, or several hundred lines of ProgramData/Microsoft/... reads as contamination.

Hash what you collect. An artefact without a hash is not evidence:

```
find evidence -type f ! -name MANIFEST.txt -exec sha256sum {} \; | sort -k2 >> MANIFEST.txt
```

Record the absent paths too. That is the difference between "this path was empty" and "we never looked".

12. Build checklist

Run this before you trust the lab with anything hostile.

```
[ ] kvm-ok reports acceleration can be used
[ ] virsh --version, virt-install --version, qemu-img --version all respond
[ ] sg libvirt -c 'virsh -c qemu:///system list --all' works
[ ] getfacl /dev/kvm shows a libvirt entry
[ ] getfacl $HOME shows user:libvirt-qemu:--x
[ ] A guest boots and virsh screenshot shows it
[ ] domblklist shows the OVERLAY, not the base
[ ] Docker internal network shows Internal=true and NO gateway
[ ] libvirt network XML contains NO <forward>
[ ] A guest/container CANNOT resolve an external name
[ ] The base image has a "clean" snapshot, taken with the guest shut down
[ ] Reset drops the overlay AND verifies it is gone
```


© Billy Forrest @ https://training.forrest.one

The isolation checks are the ones that matter. Everything above them makes the lab work; those make it safe.

13. Appendix — reproducing this guide

```
# Host evidence, captured as a real terminal transcript
docker exec -u 0 kali-desktop bash -lc '<command>' > evidence/<stamp>_<slug>.txt 2>&1

# Terminal screenshots
tools/term_shot.py images/<slug>.png <seconds> -- '<command>'
tools/crop_shots.py images/*.png

# A guest's screen, from the host, with no guest credentials
sg libvirt -c 'virsh -c qemu:///system screenshot <dom> --file /tmp/vm.ppm'
```

What is in this folder

Path	Holds
evidence/	Command transcripts from the build host
figures/	Excalidraw sources and rendered diagrams
images/	Terminal captures
scripts/	The helper scripts referenced here
tools/	Capture, crop, figure-build and PDF render tooling

Verified on this host

Component	Version
Host OS	Ubuntu 24.04.5 LTS, kernel 7.0.0-34-generic
Architecture	x86-64, 32 threads reporting VT-x
Memory	62 GiB
virsh	10.0.0
virt-install	4.1.0
qemu-img	8.2.2
Docker	29.1.3
Windows guest ISO	CESE_X64FREE_EN-US_DV9 (Enterprise LTSC eval), 4.9 GB, bootable
Kali release checked	2026.2, installer ISO 4,802,531,328 bytes

A live Windows 10 guest was built, booted and screenshotted from the host while writing this guide. The overlay/domblklist check in §6 is the one that was performed against it.

Training at Forrest.one · Building Your Own Security Lab · Built and verified on the host it documents.

Billy Forrest @ https://training.forrest.one

© Billy Forrest @ https://training.forrest.one